

PROGRAMMING 1

VARIABLE SCOPES

VARIABLE SCOPES - LOCAL

TOPICS

- **RECAP:** THE LOCAL SCOPE OF A VARIABLE
- THE GLOBAL SCOPE: WHAT + WHY

THE SCOPE OF A VARIABLE

OPERATIONS AND EXPRESSIONS

THE **SCOPE** = THE LIFETIME OF A VARIABLE (RECAP)

START

- When does it start to exist?
- Defined by position of declaration/initialisation

EXIST

- From where can I access it?
- How long does it remain in memory?
- (= consequence of the scope)

END

- When does it end its existence?

WHAT IS THE LOCAL SCOPE? (RECAP)

AND WHAT ARE THE CONSEQUENCES?

THE SCOPE OF A VARIABLE

THE LOCAL SCOPE (RECAP)

START

- DECLARE **INSIDE** A CODE BLOCK
  - code block = one or more lines of code between { }
  - For example: **inside** a function

EXIST

- CAN ONLY BE ACCESSED FROM WITHIN THAT CODE BLOCK!
  - + from nested code blocks (= code blocks **inside** that block)
  - for example: from within the function

END

- WHEN THE CODE BLOCK ENDS
  - simply said: at the closing curly bracket } from this block

Code blocks

```
void CheckAdult();

int main()
{
    CheckAdult();

    //can NOT access adult and age; local variables inside checkAdult functio
    std::cout << "The age of this person was: " << age << std::endl;
}

void CheckAdult()
{
    bool adult{ false };    //local var
    int age{ 32 };          //local var

    if (age >= 18) //ignore
    { //start nested code block
        adult = true;       //can access!
    } //end nested code block

    std::cout << "Is this person an adult? " << adult << std::endl; //can acc
}
```

Accessing a local variable

```
void CheckAdult();

int main()
{
    CheckAdult();

    //can NOT access adult and age; local variables inside checkAdult functio
    std::cout << "The age of this person was: " << age << std::endl;
}

void CheckAdult()
{
    bool adult{ false };    //local var
    int age{ 32 };          //local var

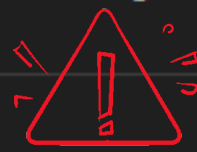
    if (age >= 18) //ignore
    { //start nested code block
        adult = true;       //can access!
    } //end nested code block

    std::cout << "Is this person an adult? " << adult << std::endl; //can acc
}
```

Error List

Entire Solution | 1 Error | 0 Warnings | 0 Messages

	Code	Description
abc	E0020	identifier "age" is undefined



DEMO TIME: ANIMATIONS

WHEN IS THE LOCAL SCOPE NOT ENOUGH?

WHAT IS THE GLOBAL SCOPE?

AND WHEN DO WE NEED IT?

THE SCOPE OF A VARIABLE

THE GLOBAL SCOPE

START

- WHEN THE PROGRAM STARTS
  - ➢ DECLARE **OUTSIDE** A CODE BLOCK
    - Console application: just after the includes
    - Framework application: game.h !

EXIST

- CAN BE ACCESSED FROM EVERYWHERE IN THAT CODE FILE!
  - for example: from within each function
  - in framework application: Start(), Draw(), Update(), …

END

- WHEN THE PROGRAM ENDS
  - simply said: when you close the program

GLOBAL VARIABLES
SO.....



WHY NOT ALL VARIABLES SHOULD BE GLOBAL

- Too many global variables creates **a mess**!!
  - Readability, reusability, maintainability
  - Think about a big game: if *all* variables are global, it will become very messy / hard to interpret!
  - Think about a function that is on its own
    - if it uses unnecessary global variables, and you want to reuse it
    - then you must not only copy the function to your other project,
    - but also search for / copy the global vars it needs

- Since they exist for the duration of your program, they take up memory that they shouldn't if not necessary

WHEN DO WE HAVE NO OTHER CHOICE?

- A variable should get the global scope if…
  - If separate functions like for example Draw(), Update(),.. need access to it (there is a caveat to this; see later)
  - If we need to ‘remember’ the value, for example.. if we want to **animate** something
    - E.g.: if something needs to keep moving right at a certain pace
    - we need to know what the position was before to add something to it
    - it needs to ‘remember’ the position